



HOW TO CREATE AN ELASTIC BEANSTALK ON AWS

Technology

December 26, 2025 Admin Leave A Comment

WHAT IS AN ELASTIC BEANSTALK: An Elastic Beanstalk application is a container for Elastic Beanstalk components, including environments where your application code runs on platforms provided and managed by Elastic Beanstalk, or in custom containers that you provide.

THROUGH UNDERSTANDING OF ENVIROMENT ELASTIC BEANSTALK

An Elastic Beanstalk environment is a collection of AWS resources running together including an Amazon EC2 instance. When you create an environment, Elastic Beanstalk provisions the necessary resources into your AWS account.

STEP 1 – CREATE AN APPLICATION **

To create your example application, you'll use the Create application console wizard. It creates an Elastic Beanstalk application and launches an environment within it.

Reminder: an environment is a collection of AWS resources required to run your application code.

**STEP 2 – CONFIGURE THE ENVIROMENT

You choose the type of application you are deploying (a Web Server environment for handling web traffic), assign an application name (Deepdivedec22app) to organize all related deployments, and optionally add tags for easier management and billing, which together define the basic structure and identity of your Elastic Beanstalk application

STEP 3 – PROCEEDURE IN CONFIGURE ENVIROMENT

In this step, you assign a unique environment name (Deepdivedec22app-env), optionally choose a custom subdomain for your app's public URL, and select the application's runtime platform (Node.js 24 on Amazon Linux 2023 with version 6.7.1), which together determine how and where your application will run in the cloud

Search

Search

RECENT POSTS

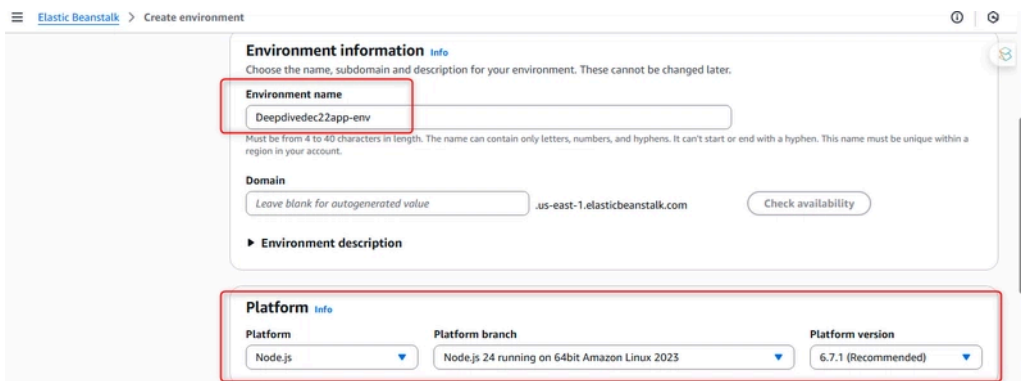
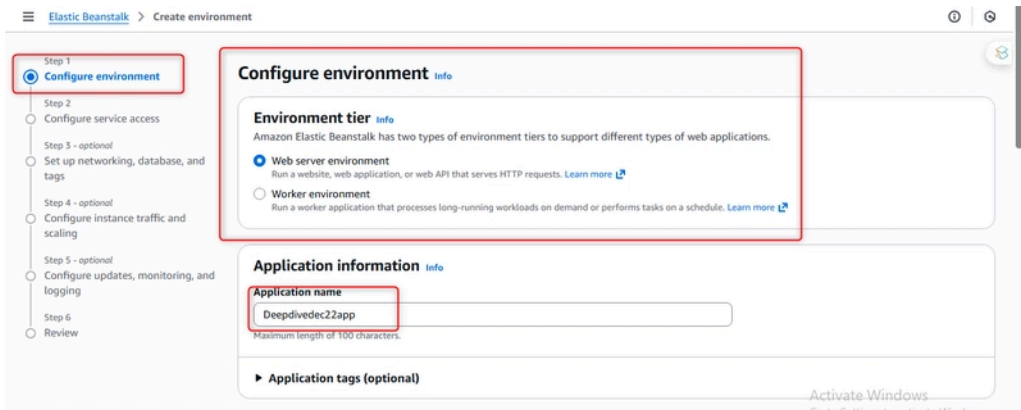
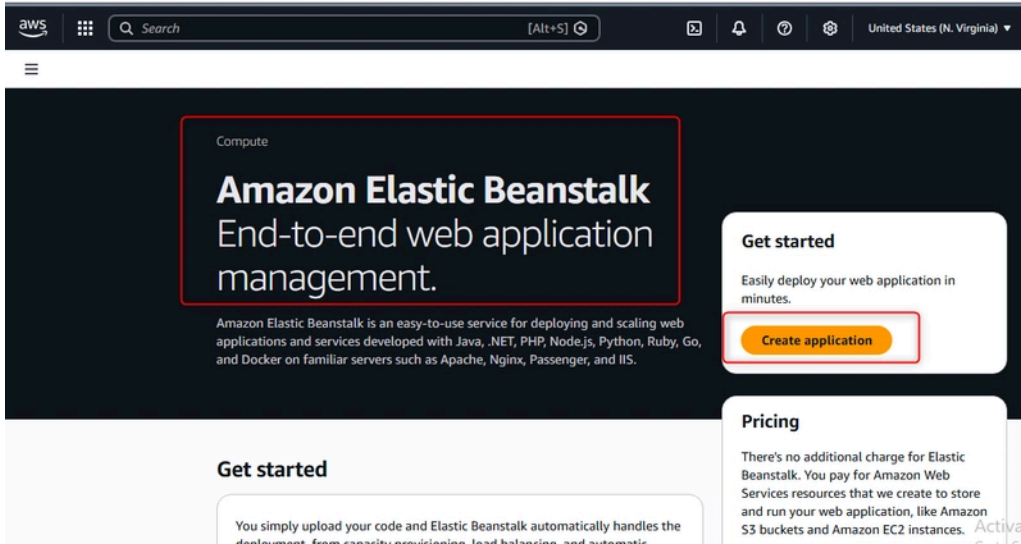
- >> What is Microsoft Planner and who is it for
- >> You need to know what the tilde (~) does in Excel
- >> To harness crypto ingenuity, financial threat must first be neutralised
- >> Ethereum-Solidity Quiz Q5: What is the Private Mempool?
- >> HOW TO CREATE AN ELASTIC BEANSTALK ON AWS

RECENT COMMENTS

admin on [What is Microsoft Planner and who is it for](#)

ARCHIVES

- December 2025
- November 2025
- October 2025
- July 2025
- June 2025



• STEP 4 NEXT PROCEDURE ON CONFIGURE ENVIROMENT *

IT is a configuration step from a cloud service deployment interface (likely AWS Elastic Beanstalk) where users must upload their application's source code and then select a pre-defined infrastructure template, or "preset," which automatically sets parameters like server type, scaling, and availability to match common deployment scenarios such as a simple test setup or a high-availability production environment.

May 2025

April 2025

March 2025

January 2025

December 2024

October 2024

September 2024

August 2024

June 2024

May 2024

April 2024

March 2024

February 2024

January 2024

December 2023

November 2023

June 2023

May 2023

April 2023

December 2022

October 2022

July 2022

June 2022

May 2022

March 2022

December 2021

April 2020

September 2019

August 2019

May 2019

Application code Info

- ☒ Sample application
- ☐ Existing version
Application versions that you have uploaded.
- ☐ Upload your code
Upload a source bundle from your computer or copy one from Amazon S3.

Presets Info

Start from a preset that matches your use case or choose custom configuration to unset recommended values and use the service's default values.

Configuration presets

- ☒ Single instance (free tier eligible)
- ☐ Single instance (using spot instance)
- ☐ High availability
- ☐ High availability (using spot and on-demand instances)
- ☐ Custom configuration

Activate Windows
 Go to Settings to activate Windows

STEP 5 CONFIGURE SERVICE ACCESS *

A configuration step for setting up service access, which is critical for security and functionality in a managed cloud service like AWS Elastic Beanstalk. Here, you must define two essential Identity and Access Management (IAM) roles: a service role that grants the Elastic Beanstalk platform itself the permissions it needs to manage resources on your behalf, and an EC2 instance profile that provides the necessary permissions to the actual application servers (EC2 instances) it launches. Additionally, there is an optional setting to select an EC2 key pair, which is used for secure SSH access to those instances for administrative purposes.

Elastic Beanstalk > Create environment

Step 1
Configure environment

Step 2
Configure service access

Step 3 - optional
Set up networking, database, and tags

Step 4 - optional
Configure instance traffic and scaling

Step 5 - optional
Configure updates, monitoring, and logging

Step 6
Review

Configure service access Info

Service access

IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create and manage your environment. Both the IAM role and instance profile must be attached to IAM managed policies that contain the required permissions. [Learn more](#)

Service role

Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.

Choose a service role [Create role](#)

EC2 instance profile

Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

Choose a instance profile [Create role](#)

EC2 key pair - optional

Select an EC2 key pair to securely log in to your EC2 instances. [Learn more](#)

Choose a key pair [Create role](#)

[Cancel](#) [Skip to review](#) [Previous](#) [Next](#)

STEP 5 HOW TO CONFIGURE SERVICE ROLE

This image Below shows the first step in creating an IAM role, where you must specify the trusted entity—the type of principal (like an AWS service, another account, or a user from an external identity provider) that is allowed to “assume” or use this role to perform actions in your AWS account. Selecting the correct trusted entity is the foundational security step that defines who or what can be granted the permissions attached to this role.

February 2019

October 2018

September 2018

December 2017

CATEGORIES

business

entertainment

general

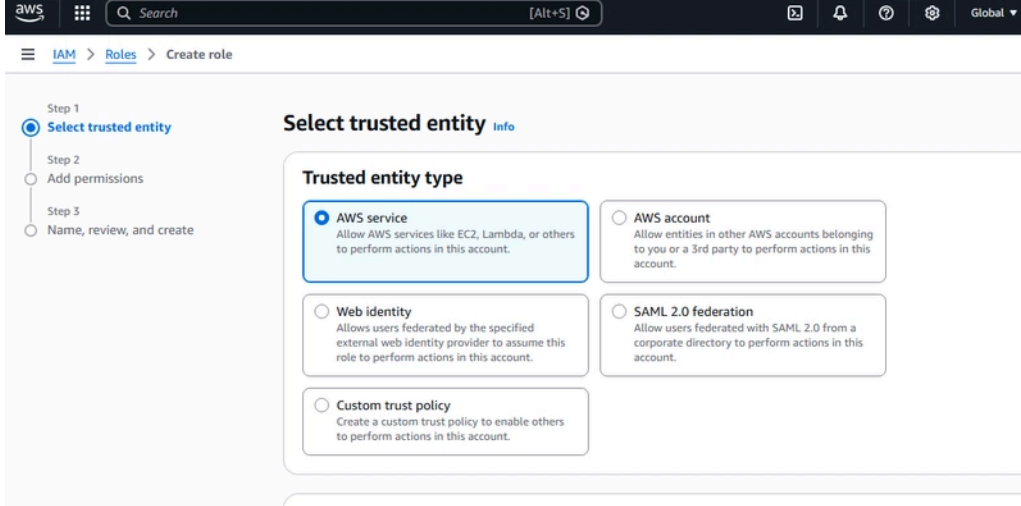
health

politics

science

sports

technology

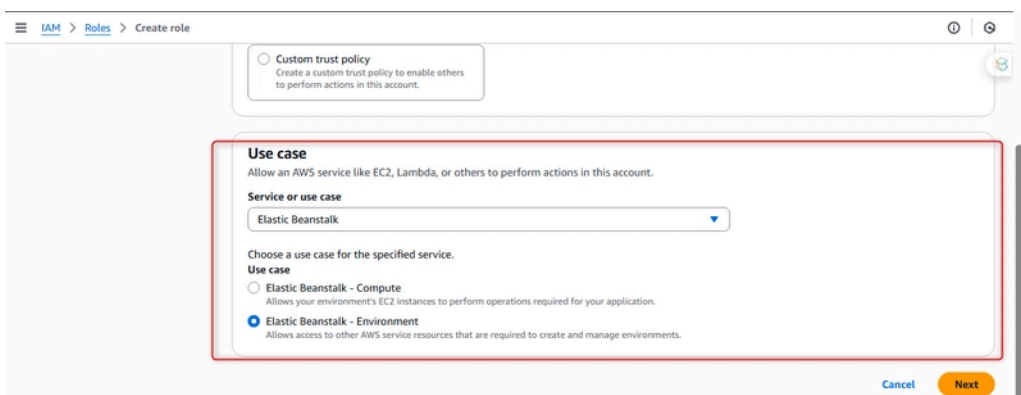


• STEP 6 ROLE ACCESS CONTINUATION *

This image Below shows the continuation of creating an IAM role for AWS Elastic Beanstalk, focusing on selecting a specific use case for the service. After choosing “Elastic Beanstalk” as the service, you are presented with two distinct use cases that define the role’s purpose.

The “Elastic Beanstalk – Compute” use case is designed for an EC2 instance profile. This role is attached to the application servers themselves, granting them permissions to perform actions necessary for the application to run, such as uploading logs to Amazon S3 or sending metrics to Amazon CloudWatch.

The “Elastic Beanstalk – Environment” use case is designed for the service role. This role is assumed by the Elastic Beanstalk platform service, giving it permission to manage AWS resources like EC2 instances, Auto Scaling groups, and load balancers on your behalf to create and manage the environment’s infrastructure. Selecting the correct use case here is crucial for ensuring the right permissions are granted to the right component of the Elastic Beanstalk architecture.



• STEP 7 ROLE ACCESS CONTINUATION *

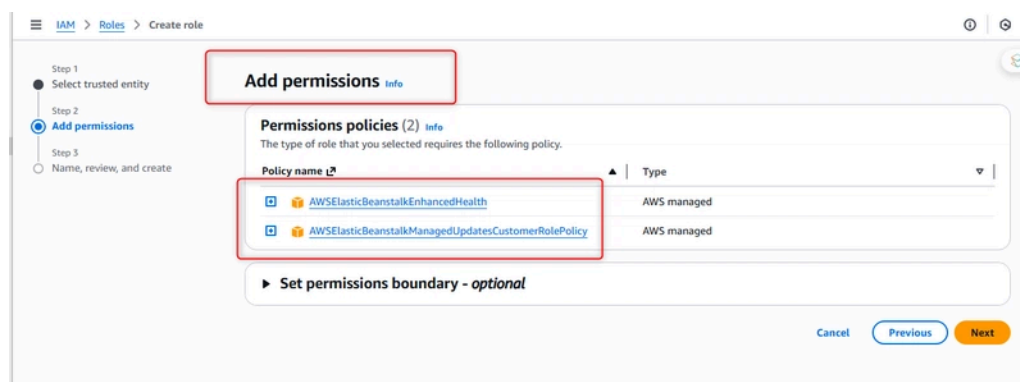
This image shows the next step in creating an IAM role for Elastic Beanstalk, where the necessary permissions policies are being attached.

When you select a specific use case (like “Elastic Beanstalk – Environment”), AWS automatically determines which managed policies are required for that role to function properly. In this case, the platform is pre-selecting two AWS-managed

policies: `AWSElasticBeanstalkEnhancedHealth` and `AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy`.

The first policy grants permissions for Elastic Beanstalk to collect and report enhanced health monitoring information from your environment. The second policy allows Elastic Beanstalk to perform managed updates on your environment, such as applying platform updates in a controlled way.

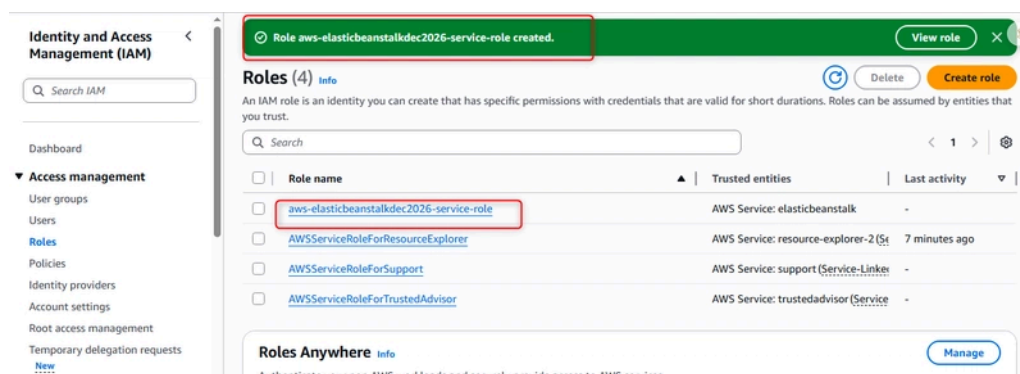
This step automates security best practices by attaching the minimum, predefined permissions needed for the role's intended function, as defined by AWS.



• STEP 8 CONTINUATION ROLE ACCESS ON SERVICE ROLE FINAL STEP *

This image shows the final step in the IAM role creation process within the AWS Management Console, confirming the successful creation of the role. The newly created role, named `aws-elasticbeanstalkdec2026-service-role`, is now visible in the list of IAM roles.

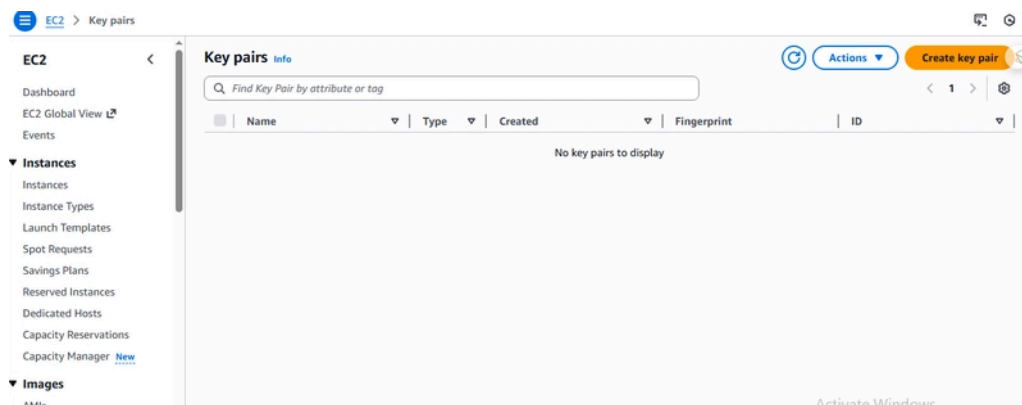
The screenshot highlights key details about the role: it is listed under “Access management,” and its trusted entity is confirmed to be the “elasticbeanstalk” AWS service, meaning only the Elastic Beanstalk platform can assume this role. The creation is marked as successful, and the role is ready to be selected in the previous “Configure service access” step, completing the setup of the required service role for deploying an environment.



STEP 9 CREATING A PAIR KEY FOR SEVICE ACCESS

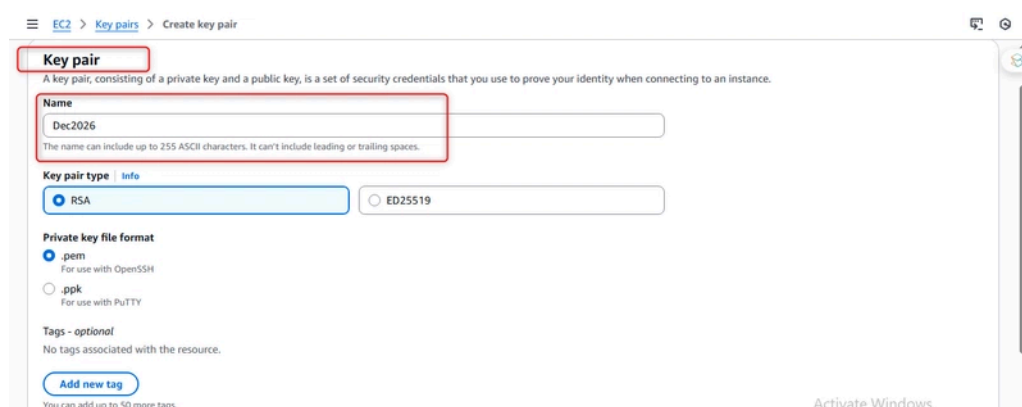
From the Amazon EC2 console, specifically the Key Pairs section. This is where you manage the SSH keys needed to securely log into your EC2 instances.

Right now, I'm looking at the list view where I can search for any existing key pairs by name, type, or other details. To make a new one, I would click the "Create key pair" button in the actions menu. You can see the rest of the EC2 navigation menu on the left, showing options for Instances, Images, and other resources, which tells me this key management is just one part of the overall process for setting up a server.



STEP 10 CONTINUATION CREATING A PAIR KEY FOR SERVICE ACCESS

Here, I need to define the key's properties. I've named it "Dec2026" and must select a Key pair type (choosing between the RSA or ED25519 encryption algorithms). I also need to choose the Private key file format—.pem for use with standard OpenSSH clients on Linux or Mac, or .ppk for use with the PuTTY client on Windows. There's also an optional section to add tags for better resource organization. Once I fill this out and create the key, the .pem or .ppk file will be automatically downloaded to my computer; this private key must be kept secure, as it's my only way to authenticate via SSH.



STEP 11 FINAL STEP FOR CONFIGURING SERVICE ACCESS

I have successfully configured all the necessary security identities. The previously created service role (aws-elasticbeanstalkdec2026-service-role) and EC2 instance profile (aws-elasticbeanstalkdec2026-ec2-role) are now selected, granting the platform and the application servers their required permissions. Additionally, I've chosen the optional EC2 key pair named "Dec2026," which will allow me to SSH into the instances for direct access if needed. With all these elements in place, I can proceed to the next optional steps for networking and scaling, or go straight to reviewing and launching the environment.

Configure service access Info

Service access
IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create and manage your environment. Both the IAM role and instance profile must be attached to IAM managed policies that contain the required permissions. [Learn more](#)

Service role
Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.

aws-elasticbeanstalk-ec2-role [Create role](#)

EC2 instance profile
Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

aws-elasticbeanstalk-ec2-role [Create role](#)

EC2 key pair - optional
Select an EC2 key pair to securely log in to your EC2 instances. [Learn more](#)

Dec2026 [Create key pair](#)

Cancel [Skip to review](#) [Previous](#) [Next](#)

• STEP 12 HOW TO SET UP NETWORKING DATABASE AND TAG *

Here, I'm configuring the network layout for my application. I've selected a custom VPC named december20026project-vpc instead of using the default one. The main decision on this screen is whether to assign a public IP address to the EC2 instances. Enabling this would allow the instances to be directly reachable from the internet, which is a key security and architectural choice. The interface also shows where I can choose specific subnets within the VPC to launch the instances into.

Set up networking, database, and tags - optional Info

Instance settings
Choose a subnet in each AZ for the instances that run your application. To avoid exposing your instances to the Internet, run your instances in private subnets and load balancer in public subnets. To run your load balancer and instances in the same public subnets, assign public IP addresses to the instances. [Learn more](#)

VPC
Launch your environment in a custom VPC instead of the default VPC. You can create a VPC and subnets in the VPC management console. [Learn more](#)

vpc-09cd464b8cd3c8561 | 10.0.0.0/16 | december20026project-vpc [Create VPC](#)

Public IP address
Assign a public IP address to the Amazon EC2 instances in your environment.

☐ Enable

Instance subnets

Filter instance subnets

☐ Availability Zone | Subnet | CIDR | Name | [Open Windows](#)

• STEP 13 CREATE A VPC *

I'm choosing the option to create "VPC and more", which will automatically generate a complete, ready-to-use network including subnets and other core resources. I've set the Name tag to december20026project, and the VPC will be named december20026project-vpc. For the IP address range, I've specified the CIDR block 10.0.0.0/16, which provides over 65,000 private IP addresses. The preview shows that this setup will automatically create four subnets across two Availability Zones (us-east-1a and us-east-1b), establishing a foundational and redundant network structure for my application.

Create VPC Info

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances. Mouse over a resource to highlight the related resources.

VPC settings

Resources to create Info
Create only the VPC resource or the VPC and other networking resources.

☐ VPC only ☒ VPC and more

Name tag auto-generation Info
Enter a value for the Name tag. This value will be used to auto-generate Name tags for all resources in the VPC.

☒ Auto-generate
december20026project

IPv4 CIDR block Info
Determine the starting IP and the size of your VPC using CIDR notation.

10.0.0.0/16 65,536 IPs
CIDR block size must be between /16 and /28.

Preview

VPC [Show details](#)
Your AWS virtual network
december20026project-vpc

Subnets (4)
Subnets within this VPC

us-east-1a

- december20026project-subnet-
- december20026project-subnet-

us-east-1b

- december20026project-subnet-
- december20026project-subnet-

Activate Windows

STEP 14 CONFIGURE INSTANCE TRAFFIC AND SCALING

Right now, I'm specifically looking at the settings for the root volume—which is basically the main hard drive for each server. Here, I can choose the type of storage, decide how big it should be in gigabytes, and set performance details like the IOPS (how many read/write operations it can handle per second) and throughput (the data transfer speed). At the bottom, I'm also setting up CloudWatch monitoring to report metrics from the servers every five minutes, so I can keep an eye on their performance.

The screenshot shows the 'Configure instance traffic and scaling' step in the AWS Elastic Beanstalk console. The left sidebar lists the steps: Step 1: Configure environment, Step 2: Configure service access, Step 3 - optional: Set up networking, database, and tags, Step 4 - optional: Configure instance traffic and scaling (selected), Step 5 - optional: Configure updates, monitoring, and logging, Step 6: Review. The main content area is titled 'Configure instance traffic and scaling - optional'. It contains three sections: 'Instances' (Configure the Amazon EC2 instances that run your application.), 'Root volume (boot device)' (Root volume type: (Container default), Size: 100 GB, IOPS: 100, Throughput: 125 MB/s), and 'Amazon CloudWatch monitoring' (Monitoring interval: 5 minute). An 'Activate Windows' watermark is visible in the bottom right corner.

The screenshot shows the 'Instance metadata service (IMDS)' and 'EC2 security groups' sections in the AWS Elastic Beanstalk console. The 'IMDSv1' section has a 'Disable' checkbox checked. The 'EC2 security groups' section shows a dropdown menu with 'sg-0e3c9b80e902aa988 | default' selected. An 'Activate Windows' watermark is visible in the bottom right corner.

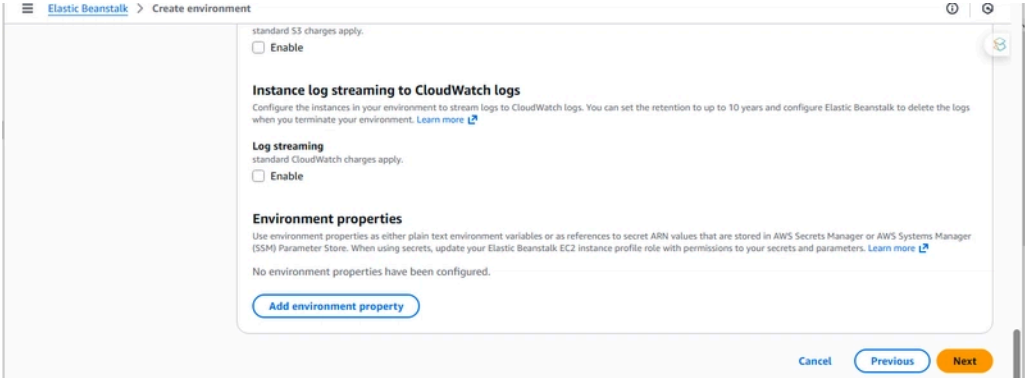
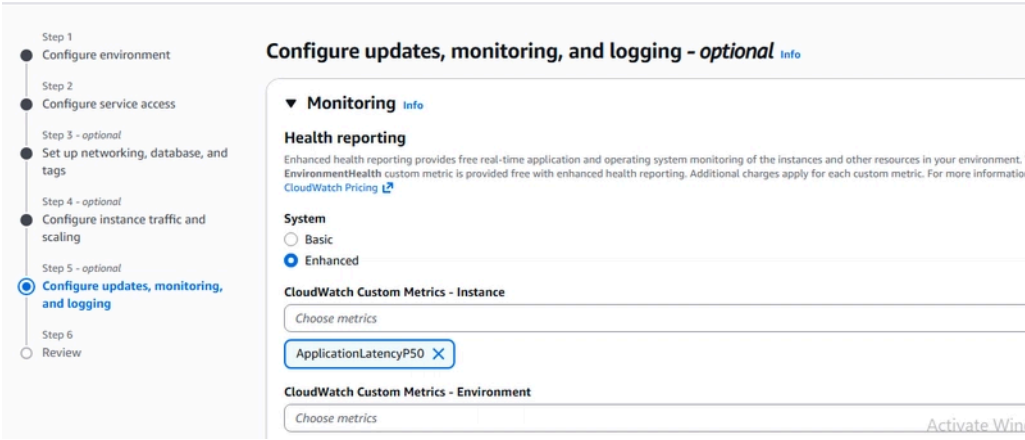
The screenshot shows the 'Instance types' and 'AMI ID' sections in the AWS Elastic Beanstalk console. The 'Instance types' section shows a list of instance types: 1. t3.micro, 2. t3.small. There are 'Add instance type' and 'Remove' buttons. The 'AMI ID' section shows the default AMI ID: ami-00bd25a5f0aaf1cf7. At the bottom, there are 'Cancel', 'Skip to review', 'Previous', and 'Next' buttons.

• STEP 15 CONFIGURE UPDATE MONITORING AND LOGGING *

I've completed all the setup steps: from the initial configuration and security access to the optional networking and server settings. This review page gives me one last chance to check every decision I've made, from the application name and platform to the chosen instance type, VPC, key pair, and monitoring options.

Scrolling through this, I can see all my configured details listed in a clear summary, including the service role, instance profile, and that I've enabled enhanced health

reporting. Seeing everything confirmed here makes me confident I’ve set it up correctly. Once I click “Submit,” Elastic Beanstalk will start provisioning all the AWS resources—like the load balancer, EC2 instances, and Auto Scaling group—to build and launch my environment.

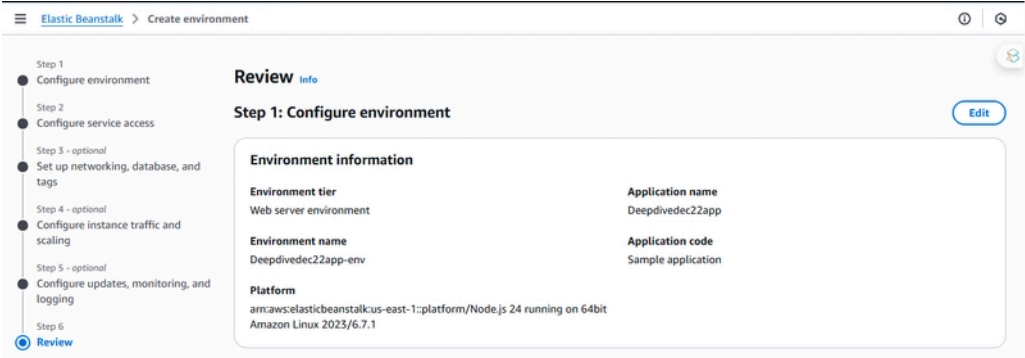


STEP 16 FINAL REVIEW BEFORE DEPLOYMENT OF APP

I’m on the final review page of the Elastic Beanstalk setup, where I’m double-checking all my configuration choices before launching the environment.

This first section of the review summarizes the core setup from Step 1. I can see that my environment, named Deepdivedec22app-env, is configured as a Web server environment. It’s set to run on the Node.js 24 platform on Amazon Linux. The application itself is named Deepdivedec22app, and for this initial deployment, I’m using the sample application code provided by AWS.

The progress bar on the right confirms that I have already configured the service access, networking, instance settings, and monitoring in the previous steps. Everything looks correct here, so I’m ready to proceed to the final launch.



Edit

aws-elasticbeanstalkde2026-ec2-role

Edit

subnet-0427814fc94a44641

There are no tags defined

No environment properties

The screenshot displays the AWS Elastic Beanstalk application console. At the top, the browser address bar shows the URL: `jan2024-env.eba-6wtmfhyd.us-east-1.elasticbeanstalk.com`. The main heading is **AWS Elastic Beanstalk**. Below this, a large green banner contains the text: **Welcome to Your Elastic Beanstalk Application**. Underneath the banner, a message reads: **Congratulations! Your Node.js application is now running on your own dedicated environment in the AWS Cloud.** A green button labeled **Learn More** is positioned below the message. At the bottom of the screenshot, a white box with a red border contains the heading **Benefits of AWS Elastic Beanstalk**, followed by the text: **Discover why thousands of developers rely on AWS Elastic Beanstalk to deploy and manage their applications.** In the bottom right corner, there is a watermark for 'Activate Windows'.

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐

Save my name,
email, and website
in this browser for
the next time I
comment.

Post Comment